Your First iOS App: A Meditation Timer

A step-by-step SwiftUI tutorial for Java/React developers. By the end you'll have a working app with a session-length picker, a start/pause timer, and a progress ring.

---

0. Setup

1. Install **Xcode** from the Mac App Store (it's large, ~15GB — start this first if you haven't).
2. Open Xcode → **Create New Project** → **iOS** → **App**.
3. Fill in:
   - Product Name: `MindfulTimer`
   - Interface: **SwiftUI**
   - Language: **Swift**
   - Uncheck "Use Core Data" and "Include Tests" for now.
4. Save it wherever you keep projects. Xcode will open with `ContentView.swift`.
5. Click the **Play** button (top-left) to build and run in the Simulator. You should see "Hello, world!"

---

1. Mental model: SwiftUI vs. React

You already know this pattern — the names just change:

React	SwiftUI
Component (function returning JSX)	`struct` conforming to `View`, with a `body` property
`useState`	`@State` property
Props	`let` properties passed into a view's initializer
JSX tree	Declarative view builder (`VStack`, `HStack`, `Text`, etc.)
CSS	Modifiers chained with `.` (e.g. `.padding()`, `.foregroundColor(.blue)`)
Conditional rendering (`{cond && <X/>}`)	`if`/`else` directly inside the view builder

Swift itself, for a Java developer: strong static typing, optionals (`String?`) instead of null-checking everywhere, and `struct` (value type, like a Java record) is the default instead of `class`.

---

2. Step 1 — Replace the boilerplate

Open `ContentView.swift` and replace everything with:

```
import SwiftUI

struct ContentView: View {
    var body: some View {
        VStack(spacing: 24) {
            Text("Mindful Timer")
                .font(.largeTitle)
                .bold()
        }
        .padding()
    }
}

#Preview {
    ContentView()
}
```

Note `#Preview` — it powers the live canvas on the right side of Xcode (View → Canvas if it's hidden). This is your fast feedback loop; you won't need to relaunch the Simulator for every small change.

---

3. Step 2 — Add state: a session-length picker

Like `useState` in React, `@State` gives a view its own local, mutable data. Add this:

```
struct ContentView: View {
    @State private var selectedMinutes: Int = 5
    let minuteOptions = [3, 5, 10, 15, 20]

    var body: some View {
        VStack(spacing: 24) {
            Text("Mindful Timer")
                .font(.largeTitle)
                .bold()

            Picker("Minutes", selection: $selectedMinutes) {
                ForEach(minuteOptions, id: \.self) { minutes in
                    Text("\(minutes) min").tag(minutes)
                }
            }
            .pickerStyle(.segmented)
        }
        .padding()
    }
}
```

The `$selectedMinutes` is a **binding** — a two-way connection, similar to passing both a value and its setter down to a controlled `<select>` in React. `ForEach` is your `.map()`.

Run it. You should see a segmented control you can tap.

---

4. Step 3 — Timer logic

Add the countdown state and a `Timer` (Swift's `setInterval`, roughly):

```
struct ContentView: View {
    @State private var selectedMinutes: Int = 5
    @State private var secondsRemaining: Int = 5 * 60
    @State private var isRunning: Bool = false
    let minuteOptions = [3, 5, 10, 15, 20]

    let timer = Timer.publish(every: 1, on: .main, in:
.common).autoconnect()

    var body: some View {
        VStack(spacing: 24) {
            Text("Mindful Timer")
                .font(.largeTitle)
                .bold()

            Picker("Minutes", selection: $selectedMinutes) {
                ForEach(minuteOptions, id: \.self) { minutes in
                    Text("\(minutes) min").tag(minutes)
                }
            }
            .pickerStyle(.segmented)
            .disabled(isRunning)
            .onChange(of: selectedMinutes) {
                secondsRemaining = selectedMinutes * 60
            }

            Text(timeString(from: secondsRemaining))
                .font(.system(size: 56, weight: .thin, design: .rounded))
                .monospacedDigit()

            Button(isRunning ? "Pause" : "Start") {
                isRunning.toggle()
            }
            .buttonStyle(.borderedProminent)
            .disabled(secondsRemaining == 0)
        }
        .padding()
        .onReceive(timer) { _ in
            guard isRunning, secondsRemaining > 0 else { return }
            secondsRemaining -= 1
        }
    }

    func timeString(from seconds: Int) -> String {
        let m = seconds / 60
```

```
        let s = seconds % 60
        return String(format: "%02d:%02d", m, s)
    }
}
```

What's new here:

- `Timer.publish(...).autoconnect()` creates a repeating publisher — think of it as a ticking clock the view subscribes to.
- `.onReceive(timer)` runs every tick, like a `useEffect` with a `setInterval` inside — except SwiftUI manages the subscription lifecycle for you.
- `.onChange(of:)` reacts to state changes, similar to a `useEffect` with a dependency array.
- String interpolation `"\(minutes) min"` is like a JS template literal.

Run it — pick a duration, hit Start, watch it count down.

---

5. Step 4 — Add a progress ring

This is where SwiftUI's declarative drawing shines. Add above the time `Text`:

```
ZStack {
    Circle()
        .stroke(lineWidth: 12)
        .opacity(0.15)

    Circle()
        .trim(from: 0, to: progressFraction)
        .stroke(style: StrokeStyle(lineWidth: 12, lineCap: .round))
        .rotationEffect(.degrees(-90))
        .animation(.linear(duration: 1), value: progressFraction)

    Text(timeString(from: secondsRemaining))
        .font(.system(size: 44, weight: .thin, design: .rounded))
        .monospacedDigit()
}
.frame(width: 220, height: 220)
```

And add the computed property (like a React derived value / getter):

```
var progressFraction: Double {
    let total = Double(selectedMinutes * 60)
    guard total > 0 else { return 0 }
    return Double(secondsRemaining) / total
}
```

`ZStack` layers views on top of each other (like `position: absolute` children). `.trim` draws a partial arc of the circle, and animating it gives you a smooth ring that drains down like sand in an hourglass.

Remove the standalone time `Text` you added in Step 3 since it now lives inside the `ZStack`.

---

6. Step 5 — Polish (optional, but satisfying)

A few small additions that make it feel like a real app:

- **Reset on finish:** in the `.onReceive(timer)` block, when `secondsRemaining` hits 0, set `isRunning = false`.
- **Haptic feedback:** `UINotificationFeedbackGenerator().notificationOccurred(.success)` when the session ends.
- **Background color:** wrap the whole `VStack` in a `ZStack` with `Color(.systemIndigo).opacity(0.05).ignoresSafeArea()` behind it for a calmer look.
- **App icon & name:** click the blue project icon in the navigator → General tab → set Display Name; drag icon images into `Assets.xcassets` → AppIcon.

---

7. Where to go next

- **Add your own audio:** `AVAudioPlayer` can loop a bundled meditation track while the timer runs — a natural next step given what you already compose.
- **Persistence:** `@AppStorage` (same idea as `@State`, but backed by `UserDefaults`) to remember the last-picked duration between launches.
- **Notifications:** `UNUserNotificationCenter` to fire a local notification if the app is backgrounded when the session ends.
- **Apple's official tutorial**, "Introducing SwiftUI," is a good second stop once this feels comfortable — it covers navigation and lists, which this project intentionally skips.

Take this one step at a time in Xcode — run after each section rather than pasting the whole thing at once, so you can see what each addition actually does.